

Chapter 1

Dynamic Programming

Contents

1	Introduction	2
2	Policy Evaluation	2
3	Policy Improvement	2
4	Obtaining Optimal Policy	4
4.1	Policy Iteration	4
4.2	Value Iteration	4
5	Asynchronous Dynamic Programming	5
6	Generalized Policy Iteration	5

1 Introduction

Dynamic programming (DP) refers to a collection of algorithms which finds out optimal policies given a *perfect* model of MDP (the dynamics of MDP $p(s', a|s, a)$). Since it requires perfect MDP model and has expensive computational cost, DP is not used so much in real applications. However, the methods shown later (Monte Carlo, Temporal Difference) are all rooted from dynamic programming. DP is a foundation of these methods.

2 Policy Evaluation

First, we consider how to compute the state-value function v_π when a policy π is given. Using the Bellman equation, we can get an *iterative policy evaluation algorithm*. Starting with initial state-value function v_0 , we update the state-value function as following;

$$v_{k+1}(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_k(S_{t+1}|S_t = s)] \quad (1.1)$$

$$= \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a)[r + \gamma v_k(s')] \quad (1.2)$$

for all $s \in \mathcal{S}$. We can see that $v_k = v_\pi$ is the fixed point for this update rule. The sequence $\{v_k\}$ is known to converge to v_π under the same condition that guarantee the existence of v_π . The algorithm pseudocode is given as follows.

Algorithm 1: Iterative Policy Evaluation Algorithm

Input : $p(s', r|s, a)$ the dynamics of given finite MDP

π given policy

Output: v_π the state-value function

```
1  $\theta \leftarrow$  appropriate threshold value for convergence
2 initialize  $V(s)$  for all  $s \in \mathcal{S}^+$  except that  $V(\text{terminal}) = 0$ 
3 repeat
4    $\Delta \leftarrow 0$ 
5   foreach  $s \in \mathcal{S}$  do
6      $v \leftarrow V(s)$ 
7      $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$ 
8      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
9 until  $\Delta < \theta$ 
10 return  $V \simeq v_\pi$ 
```

One can notice that the algorithm 1 is not really same as the update equation 1.2. When updating the value of state, we may use the values of states that are newly updated, unlike the equation 1.2. However, this is also known to converge well, even more quickly than when only using the old values of states. This is intuitively reasonable since the new values can be regarded more closer to the real value.

3 Policy Improvement

Now we have evaluated the value function. Our next step is to improve the policy with the state-value function we computed. The improvement of the policy has its root on the *policy improvement theorem*:

Theorem 3.0.1 (Policy Improvement Theorem)

Let π , v_π and q_π be a given deterministic policy and state-value function and action-value function of the policy π , respectively. Let π' be another deterministic policy. Then, the following holds:

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \Rightarrow v_{\pi'}(s) \geq v_\pi(s) \text{ for all } s \in \mathcal{S} \quad (1.3)$$

Moreover, if there is a strict inequality of the equation 1.3 of left-hand side at any state, then there must be strict inequality of the right-hand side of the equation 1.3 at that state.

The idea behind the proof of this statement is quite straightforward. We let the policy π' to be identical to π except that $\pi'(s) = a \neq \pi(s)$. It is given as follows:

$$\begin{aligned} v_\pi(s) &\leq q_\pi(s, \pi'(s)) \\ &= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = \pi'(s)] \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s] \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1}) | S_t = s)] \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma E_{\pi'}[R_{t+2} + \gamma v_\pi(S_{t+2} | S_{t+1}, A_{t+1} = \pi'(S_{t+1})) | S_t = s]] \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) | S_t = s] \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_\pi(S_{t+3}) | S_t = s] \\ &\vdots \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots | S_t = s] \\ &= v_{\pi'}(s). \end{aligned}$$

Now, considering the changes at all states and to all possible actions, we consider the new *greedy* policy π' given by

$$\pi'(s) = \arg \max_{a \in \mathcal{A}(s)} q_\pi(s, a) \quad (1.4)$$

$$= \arg \max_{a \in \mathcal{A}(s)} \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = a] \quad (1.5)$$

$$= \arg \max_{a \in \mathcal{A}(s)} \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')] \quad (1.6)$$

This process, making the original policy greedy with respect to the value function of the original policy, is called *policy improvement*.

It is guaranteed for a policy to strictly improve unless the original policy is already optimal; Let π' be a policy that is as good but not better than π . Then, $v_\pi = v_{\pi'}$ and from the equation 1.5, it follows that for all $s \in \mathcal{S}$,

$$\begin{aligned} v_{\pi'}(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_{\pi'}(S_{t+1}) | S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi'}(s')]. \end{aligned}$$

And this is same as the Bellman optimal equation, and $v_{\pi'}$ must be v_* , the optimal value function, and so is v_π .

4 Obtaining Optimal Policy

4.1 Policy Iteration

Now, we can repeat the above two processes, policy evaluation and policy improvement, to obtain the optimal policy and optimal state-value function. Specifically, we start with initial policy π_0 and evaluate the state-value function v_{π_0} . Then we improve policy π_0 with v_{π_0} to get π_1 , and evaluate next state-value function v_{π_1} . And we repeat it until we get π_* and v_* .

This way of finding optimal policy is called *policy iteration*.

Since the finite MDP has only finite number of policies, this process must converge to an optimal policy in finite number of iterations.

Algorithm 2: Policy Iteration (with iterative policy evaluation)

Input : $p(s', r|s, a)$ the dynamics of given finite MDP
 π initial policy
 v initial state-value function
Output: π_* optimal policy
 v_* optimal state-value function

```

1  $\theta \leftarrow$  appropriate threshold value for convergence
2  $\Delta \leftarrow 0$ 
3 policy-stable  $\leftarrow$  false
4 repeat
    // Policy Evaluation
5   repeat
6      $\Delta \leftarrow 0$ 
7      $V_{old}(s) \leftarrow V(s)$ 
8     foreach  $s \in \mathcal{S}$  do
9        $v \leftarrow V(s)$ 
10       $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$ 
11       $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
12   until  $\Delta < \theta$ 
    // Policy Improvement
13   policy-stable  $\leftarrow$  true
14   foreach  $s \in \mathcal{S}$  do
15      $\pi'(s) \leftarrow \pi(s)$ 
16      $\pi(s) \leftarrow \arg \max_{a \in \mathcal{A}(s)} \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$ 
17     if  $\pi'(s) \neq \pi(s)$  then
18       policy-stable  $\leftarrow$  false
19 until policy-stable is true and  $\max |V_{old}(s) - V(s)| < \theta$ 

```

4.2 Value Iteration

One drawback of policy iteration is that it involves policy evaluation, which may take a long time if the state space is too large. The good news is that the policy evaluation step in policy iteration can be truncated in several ways, without

losing a convergence guarantees of policy iteration. One of such a way is *value iteration*. The value iteration combines policy improvement and truncated policy evaluation steps;

$$v_{k+1} = \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a] \quad (1.7)$$

$$= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')] \quad (1.8)$$

The sequence $\{v_k\}$ is known to converge to v_* .

Algorithm 3: Value Iteration

Input : $p(s', r | s, a)$ the dynamics of given finite MDP
 $V(s)$ initial value function, except that $V(\text{terminal}) = 0$
Output: π final policy

```

1  $\theta \leftarrow$  appropriate threshold value
2 repeat
3    $\Delta \leftarrow 0$ 
4   foreach  $s \in \mathcal{S}$  do
5      $v \leftarrow V(s)$ 
6      $V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$ 
7      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
8 until  $\Delta < \theta$ 
9 foreach  $s \in \mathcal{S}$  do
10   $\pi(s) \leftarrow \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$ 
11 return  $\pi$ 

```

Faster convergence can often be achieved by interposing multiple policy evaluation sweeps between each policy improvement step.

5 Asynchronous Dynamic Programming

As one can see, the DP methods requires a sweep along the state space for policy evaluation. This can be expensive, if the given state space is too large.

Asynchronous DP algorithms do not sweep along the state space. The values are updated without any order in this algorithms, and even some states can be updated multiple times while other states are not. However, it cannot ignore the states; for the asynchronous DP algorithms to converge, they must visit all states.

The point is that we do not have to wait for policy evaluation step to finish. We can improve our policy without sweeping a state space for policy evaluation. We might even try to skip updating some states if it is irrelevant to optimal behavior.

Asynchronous DP algorithms can easily be mixed up with real-time interaction.

6 Generalized Policy Iteration

Policy iteration involves two simultaneous interacting processes, the policy evaluation and policy improvement. In policy iteration method, these two processes wait each other to be finished. But, this is not necessary, and the example of it is

the value iteration. In asynchronous DP algorithms, these two processes intervene in much finer manner. We use the term **general policy iteration** to denote the general idea of letting policy evaluation and improvement processes interact.